

PRAKTIKUM PEMMOB 2



OLEH :

Andrew Novan Then
20240801024
Teknik Informatika

UNIVERSITAS ESA UNGGUL
TANGERANG
2026/2027

LAPORAN PEMOGRAMAN MOBILE

PERTEMUAN 3

A. MODUL

1. Input dasar dengan Text Field

TextField digunakan untuk menerima input pengguna. TextEditingController digunakan untuk membaca teks, dan dispose() dipanggil saat halaman tidak lagi digunakan.

```
final TextEditingController _controller = TextEditingController();
String _salam = '';

ElevatedButton(
  onPressed: () {
    setState(() {
      _salam = 'Halo, ${_controller.text}!';
    });
  },
  child: const Text('Sapa'),
)
```

2. Form Pendaftaran

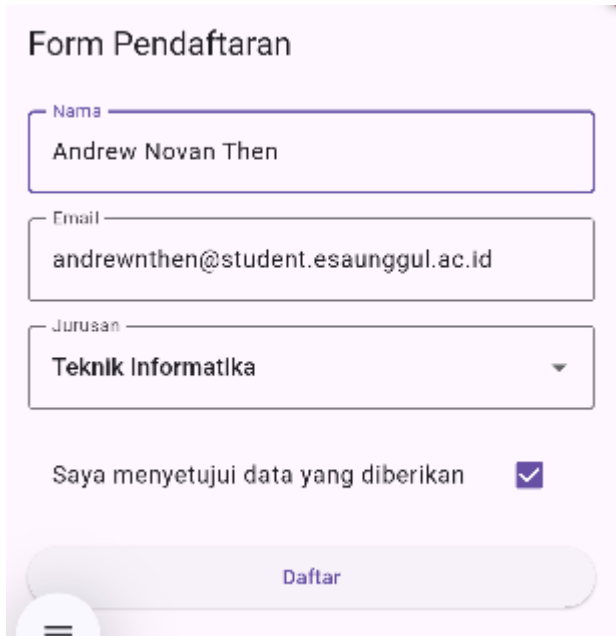
Form digunakan untuk beberapa isian yang perlu divalidasi. Praktikum menggunakan nama, email, dropdown jurusan, checkbox persetujuan, dan tombol Daftar.

```
class Makanan {
  final String nama;
  final int harga;
  const Makanan(this.nama, this.harga);
}

const daftarMenu = [
  Makanan('Nasi Goreng', 15000),
  Makanan('Mie Ayam', 12000),
  Makanan('Es Teh', 4000),
  Makanan('Ayam Bakar', 20000),
];

ListView.builder(
  itemCount: daftarMenu.length,
  itemBuilder: (context, index) {
    final item = daftarMenu[index];
    return Card(
      child: ListTile(
        leading: const Icon(Icons.restaurant),
        title: Text(item.nama),
        subtitle: Text('Rp ${item.harga}'),
        trailing: const Icon(Icons.chevron_right),
      ),
    );
  },
)
```

Hasil:

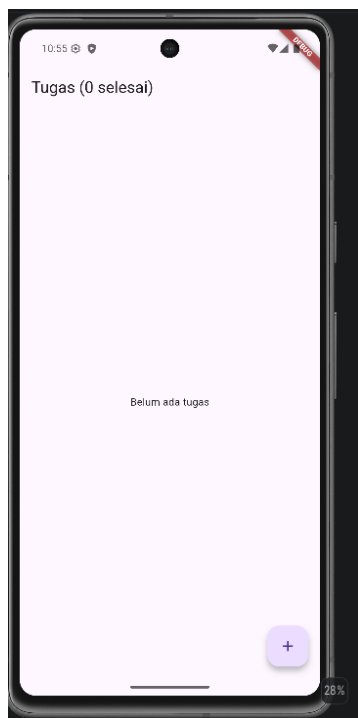


3. State Management dngan Provider

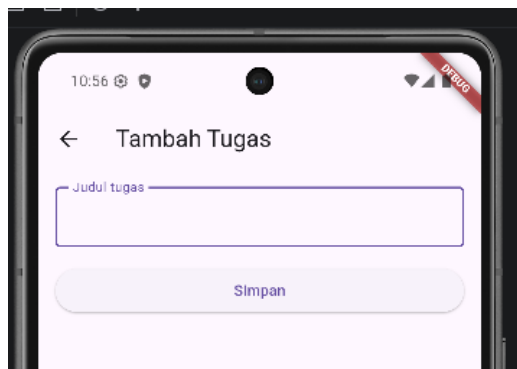
setState cukup untuk state lokal dalam satu halaman. Ketika data perlu digunakan oleh beberapa halaman, data dipindahkan ke objek bersama menggunakan ChangeNotifier dan Provider. context.watch mengikuti perubahan data, sedangkan context.read digunakan untuk aksi/callback.

```
class TugasModel extends ChangeNotifier {  
  final List<Tugas> _items = [];  
  
  List<Tugas> get items => List.unmodifiable(_items);  
  
  int get jumlahSelesai =>  
    _items.where((item) => item.selesai).length;  
  
  void tambah(String judul) {  
    _items.add(Tugas(judul));  
    notifyListeners();  
  }  
  
  void toggle(int index) {  
    _items[index].selesai = !_items[index].selesai;  
    notifyListeners();  
  }  
  
  void hapus(int index) {  
    _items.removeAt(index);  
    notifyListeners();  
  }  
}
```

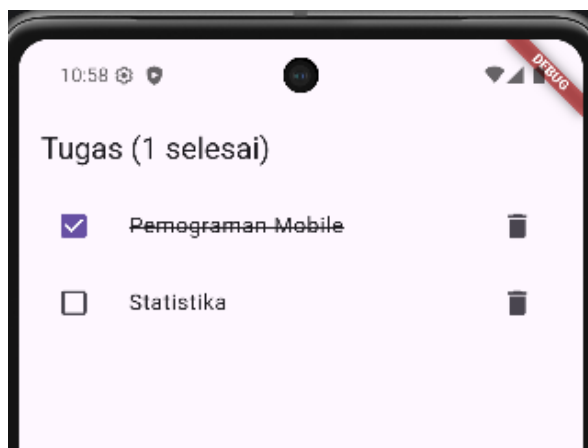
Hasil:



Tampilan saat menambahkan tugas:



Tampilan list tugas:



4. Aplikasi daftar tugas

Halaman daftar menampilkan tugas, checkbox untuk menandai selesai, ikon hapus, dan tombol tambah. Halaman Tambah Tugas menggunakan form, validasi minimal 3 karakter, Snackbar setelah disimpan, dan pesan ketika daftar kosong.

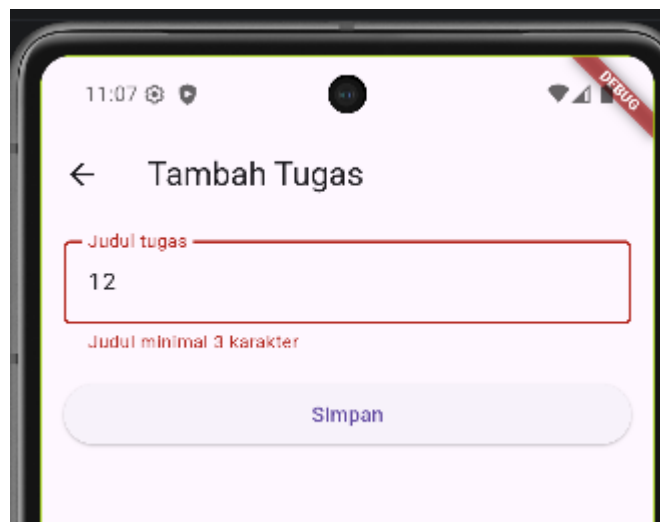
```
final model = context.watch<TugasModel>();  
  
context.read<TugasModel>().tambah(judul);  
context.read<TugasModel>().toggle(index);  
context.read<TugasModel>().hapus(index);
```



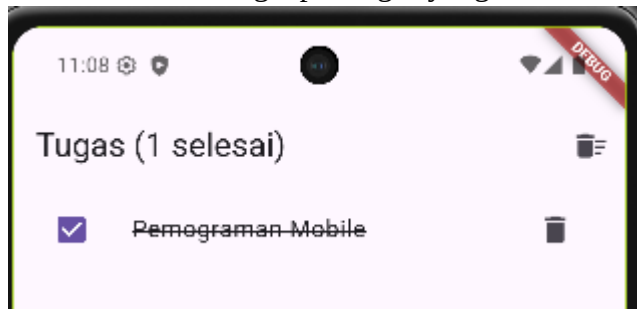
B. LATIHAN MANDIRI

1. Validasi judul pada TambahPage minimal 3 karakter.
2. Menambahkan hapusSelesai() dan tombol AppBar untuk menghapus tugas selesai.
3. Menampilkan Snackbar 'Tugas ditambahkan' setelah tugas disimpan.
4. Menampilkan 'Belum ada tugas' ketika daftar kosong.

1. Minimal huruf harus 3 karakter

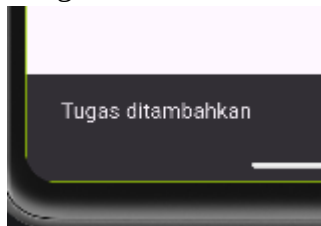


2. Tombol untuk menghapus tugas yang selesai

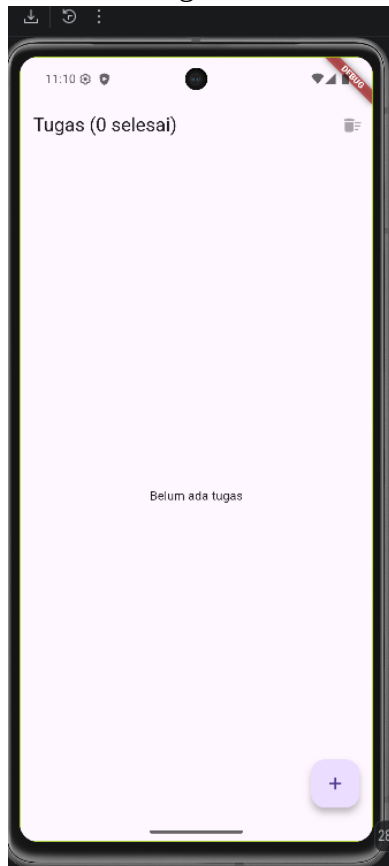


Klik hapus bagian kanan tugas jika ingin menghapus 1 tugas, dan pojok kanan atas langsung otomatis menghapus semua tugas yang tercentang / sudah selesai

3. “Tugas ditambahkan” saat menambahkan tugas baru



4. Belum ada tugas saat daftar tugas masih kosong



C. TUGAS DAFTAR BELANJA

Buat aplikasi Daftar Belanja dengan form nama barang, jumlah lebih dari 0, dan kategori dropdown dengan validasi. Halaman daftar menampilkan semua barang, dapat dicentang sebagai sudah dibeli dan dihapus. State disimpan dalam satu ChangeNotifier dan dibagikan menggunakan Provider. AppBar menampilkan jumlah barang yang belum dibeli.

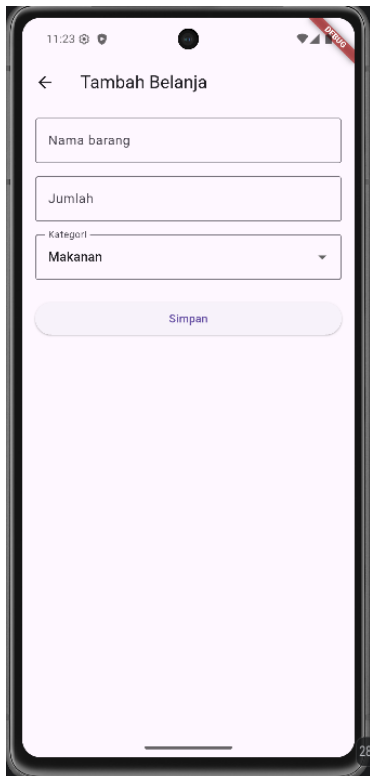
```
class Belanja {  
    final String nama;  
    final int jumlah;  
    final String kategori;  
    bool dibeli;  
  
    Belanja(this.nama, this.jumlah, this.kategori,  
        {this.dibeli = false});  
}  
  
class BelanjaModel extends ChangeNotifier {  
    final List<Belanja> _items = [];  
  
    List<Belanja> get items => List.unmodifiable(_items);  
  
    int get jumlahBelumDibeli =>  
        _items.where((item) => !item.dibeli).length;  
  
    void tambah(String nama, int jumlah, String kategori) {  
        _items.add(Belanja(nama, jumlah, kategori));  
        notifyListeners();  
    }  
}
```

Membuat aplikasi daftar belanja

Halaman utama:

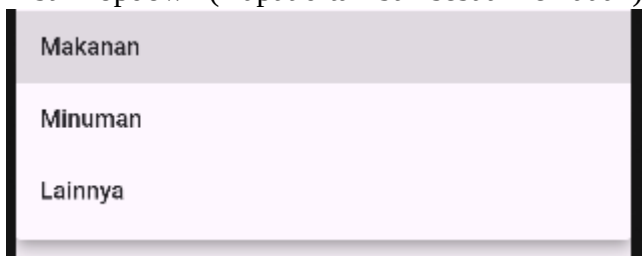


Halaman tambah atau beli barang:



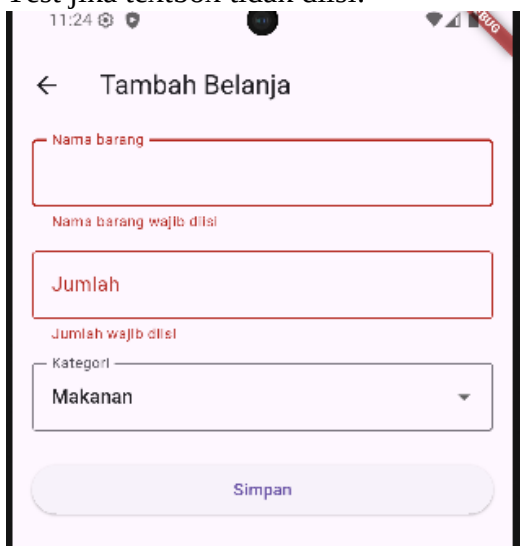
The screenshot shows a mobile app interface for adding a shopping item. At the top, the status bar displays the time 11:23 and various icons. The app header shows a back arrow and the title 'Tambah Belanja'. Below the header, there are three input fields: 'Nama barang' (empty), 'Jumlah' (empty), and 'Kategori' (a dropdown menu currently showing 'Makanan'). At the bottom of the form is a light blue button labeled 'Simpan'. The background is a light pink color.

List Dropdown (Dapat ditambah sesuai kemauan):



The screenshot shows a vertical list of three options: 'Makanan', 'Minuman', and 'Lainnya'. The 'Makanan' option is highlighted with a grey background, while the other two have a light pink background. This represents the dropdown menu from the previous form.

Test jika textbox tidak diisi:



The screenshot shows the same 'Tambah Belanja' form, but now with validation errors. The 'Nama barang' and 'Jumlah' textboxes are outlined in red. Below each red outline is a red error message: 'Nama barang wajib diisi' and 'Jumlah wajib diisi' respectively. The 'Kategori' dropdown still shows 'Makanan'. The 'Simpan' button remains at the bottom. The background is light pink.

Tampilan utama saat barang sudah ditambahkan:

